# REPLACING REPOSITORY EXPLORATION WITH TYPED JEV DECISIONS
# CS329Z MIDWAY REPORT

**Kristjan Dagur Egilsson** **Ómar Ingi Halldórsson** **Rami Ratl Mrad**

Code, protocols and run archives:
`https://github.com/KristjanDagur/CS329zFinalProject`

## 1 RESEARCH QUESTIONS

Coding agents spend much of their budget on routine decisions. We ask whether a structured decision model can replace one narrow component of an existing coding agent at lower total cost and latency while preserving bug-repair success. Following the instructor's advice to obtain mechanical evidence inside an existing harness before redesigning one, we replace the repository-exploration subagent of OpenCode with deterministic file retrieval plus one typed Jev decision that chooses which files the agent should inspect; the same strong model still writes and tests every patch. We address three questions. (Q1) Does the substitution preserve repair success while reducing cost and wall time? (Q2) Does Jev choose relevant files better than retrieval alone, and how does it compare with general LLMs given the same candidates? (Q3) Does better file choice translate into better repairs, and under what conditions? Repair success is graded by each issue's own regression tests; cost is the billed total of all model calls; time is agent wall time.

## 2 ENVIRONMENT & DATA

**Harness and tools.** We use the pinned OpenCode 1.19.5 command-line harness with its native build agent, `task` delegation tool, `explore` subagent and file tools (OpenCode contributors, 2026). Each rollout starts from an exact upstream snapshot without git history. The issue prompt asks the build agent (the *writer*) to delegate localization to the explorer before editing, then implement a minimal fix and optionally run existing tests; it is identical across conditions. The writer keeps its own `read`, `grep` and `glob` tools and may edit only implementation files, never tests. These are tool permissions, not an operating-system sandbox.

**Tasks.** All tasks are real GitHub issues with upstream fixes and regression tests from SWE-bench (Jimenez et al., 2024); no task was invented. (i) *Development pilot*: the five pydicom issues of the SWE-bench Lite development split (SWE-bench contributors, 2026a). (ii) *Extended development run*: the remaining 51 pydicom issues of the full SWE-bench development split, run with and without a restricted writer; 41 pass our environment controls and 10 are dropped and listed because even their upstream fix fails in the pinned environment (removed NumPy APIs, Python 3.12 behaviour). (iii) *Verified pilot*: ten tasks from SWE-bench Verified, a human-screened 500-task subset of the test split (OpenAI, 2024; SWE-bench contributors, 2026b) (five Django, two SymPy, one each from Matplotlib, scikit-learn and Sphinx), drawn by stratified hash ordering from a frozen 50-task sample using only instance IDs. (iv) *File-selection study*: all 225 issues of the SWE-bench development split across six repositories. Every task list, base commit and protocol was committed before any outcome was observed. The SWE-bench test split outside the Verified sample is reserved for held-out final evaluation. Public issues may appear in model training data; we make no contamination-free claim.

**Evaluation and success criteria.** Development patches are applied with the benchmark's test patch to a fresh snapshot and graded by the published fail-to-pass and pass-to-pass tests under pinned local pytest; before any paid run every environment is validated (the broken base fails, the upstream fix passes). This local grading is not an official SWE-bench score. Verified patches are graded by the official `swebench` 5.0.2 Docker harness; every image was checked to contain no test patch, gold fix or later upstream history, and every gold fix resolves. For file selection, an issue's *gold files* are

the implementation files its upstream fix modifies; the primary metric is *hit@3*, a gold file among the three chosen files, with candidate recall (a gold file among the 12 candidates) as its ceiling. Gold assets are read only by graders and scorers, never by the agent or Jev.

## 3 METHODS

**One internal substitution.** The writer is `openai/gpt-6.1-sol` in every condition. A thin OpenAI-compatible adapter exposes separate build and explore endpoints, so OpenCode's own `task` tool still creates the explorer session and returns its report to the writer. In the *native* condition the explorer is a GPT-6.1-Sol subagent with read-only tools. In the *Jev* condition only the explorer's model call is replaced: a deterministic BM25-style search over implementation files ranks candidates against the writer's exploration request, the top 12 with their most query-matching 16-line windows go to Jev 1.14, which answers one Noul relevance question per file in a single call (OpenRouter, 2026), and the three highest-scoring files are returned. Jev writes no code and selects no model; on an API or validation error the native explorer is used and both costs are counted. Scores are not calibrated probabilities and no threshold is applied. The *search* control returns the top three retrieval candidates without Jev, testing whether any benefit needs the decision model. The extended run adds an *oracle* that returns the files the upstream fix changes (paths and snippets only), an upper bound on what better localization can buy.

**File-selection study and sweep.** The offline study reuses the identical retrieval and Jev question, generalized to each repository's package directory; it reproduces the development pilot's ten archived candidate lists exactly. Its query is the issue text rather than the writer's exploration request. A separately frozen sweep repeats it with 24 and 48 candidates. As general-LLM baselines, GPT-6.1-Sol and gpt-6-luna (low reasoning effort) receive exactly Jev's request, the same 12 stored candidates and Jev's twelve questions, answer one probability per file, and are scored with Jev's top-3 rule.

**Restricted writer.** To test whether selection quality matters when the writer cannot recover by itself, a further frozen protocol reruns Jev, search and the oracle on the same 41 issues with the files chosen from the issue text before launch; the writer may then read and edit only those files, `grep` and `glob` are denied, and it may still run tests. Native exploration is omitted because its files are known only mid-run. One amendment, recorded before any outcome, gives each run its own OpenCode state directory after a shared session database stalled the first smoke run.

**Limits, accounting and timing.** All conditions share 16 writer steps, 6 explorer steps, at most 32 requests, 4,096 output tokens per request and 480 s per run; the Verified runner refuses to run if any shared setting differs. The adapter records every billed response; durable ledgers cap spending per run and per experiment, and the coding process never receives the real API key. The extended run splits time into exploration (retrieval plus the Jev decision, or the native explorer's calls), writer calls and grading. Conditions run in four parallel lanes with per-issue shuffled order, so load affects all conditions alike. Every run archives its protocol hash, trace, receipts, file decisions, patch and test log.

## 4 SUMMARY OF PROGRESS

**Q1: the substitution saves a third of time and cost without changing success.** In the extended run (Table 2), Jev changed mean agent time by $-16.3$ s per issue relative to native (-35%; 95% paired bootstrap interval $-19.1$ to $-13.7$ s) and cost by -35%. Almost all of the saving is exploration (21.6 s of model calls for native, 1.6 s for retrieval and one decision); writer time is similar across conditions. Success did not differ: 18, 19, 21 and 20 of 41 issues for native, Jev, search and oracle, with at most three discordant issues per pair. The pilots agree (Table 1): on Verified, native and Jev each resolved 9/10 and search 8/10, while cost fell from \$0.106 to \$0.056 (48%) and time from 42.9 to 26.0 s; Jev's ten decisions replaced 48 explorer generations. No run timed out, fell back or made a forbidden edit.

Table 1: Development and Verified pilots, one rollout per task and condition; cost includes all model calls; time is agent wall time excluding grading. Bottom: mean cost split into the explore step and the writer.

| Condition | Solved | Cost (USD) | Explore | Writer | Time (s) |
|---|---|---|---|---|---|
| *Development pilot (5 pydicom issues, local tests)* | | | | | |
| Native generative explorer | 2/5 | 0.110 | | | 37.6 |
| Jev file selection | 2/5 | 0.073 | | | 25.9 |
| Search only | 2/5 | 0.062 | | | 24.9 |
| *Verified pilot (10 tasks, 5 repositories, official harness)* | | | | | |
| Native generative explorer | 9/10 | 0.106 | 0.0572 | 0.049 | 42.9 |
| Jev file selection | 9/10 | 0.056 | 0.0002 | 0.056 | 26.0 |
| Search only | 8/10 | 0.054 | 0.0000 | 0.054 | 25.3 |

Table 2: Extended development run: 41 pydicom issues, one rollout per issue and condition, local tests. Explore: the exploration part of mean wall time (for the restricted writer, the file choice before launch). Last column: the explorer's report named a gold file.

| Condition | Solved | Cost (USD) | Time (s) | Explore (s) | Gold named (%) |
|---|---|---|---|---|---|
| *Free writer (own read, grep and glob tools)* | | | | | |
| Native generative explorer | 18/41 | 0.1390 | 46.5 | 21.6 | 98 |
| Jev file selection | 19/41 | 0.0910 | 30.2 | 1.6 | 93 |
| Search only | 21/41 | 0.0868 | 28.7 | 1.4 | 63 |
| Oracle (gold files) | 20/41 | 0.0908 | 27.4 | 0.0 | 100 |
| *Restricted writer (may read and edit only the chosen files)* | | | | | |
| Jev file selection | 14/41 | 0.0844 | 46.3 | 1.6 | 88 |
| Search only | 11/41 | 0.0943 | 53.1 | 1.4 | 68 |
| Oracle (gold files) | 16/41 | 0.0848 | 45.2 | 0.0 | 100 |

**Q2: Jev chooses files better than search, as well as a cheap LLM, and much faster.** Across 225 issues (Table 3), Jev placed a gold file among its three choices for 85.3% of issues, against 70.2% for search and 30.8% for a random choice among the same candidates: a paired gain of 15.1 points (95% interval 9.8 to 20.4; 38 wins, 4 losses, sign test $p < 0.001$), consistent in every repository. On identical input (Table 4), gpt-6-luna matched Jev (0.4 points, interval -1.8 to 2.7; 4/3 wins/losses) and GPT-6.1-Sol was slightly better (2.7 points, interval 0.4 to 5.3; 7/1), but the LLMs took 5–6 times as long per decision and Sol cost 58 times as much. When retrieval found a gold file, Sol chose one 99.0% of the time and Jev 96.0%, so retrieval recall (88.9%) is the main ceiling. Longer shortlists lift it: at 24 candidates Jev recovered 13 of the 25 issues whose gold file missed the 12 and kept 189 of 192 already solved (gain 4.9 points, interval 1.3 to 8.4, $p = 0.013$); at 48, 44 of 225 decisions took about 8 s. All selection studies cost $3.27.

**Q3: better selection does not yet change repairs, because the writer recovers.** Inside the agent, Jev's report named a gold file far more often than search's (Table 2), yet Jev and search did not differ in success, time (1.4 s, interval $-0.2$ to 3.1) or cost (5%). On the 13 issues where only Jev's report contained a gold file, both repaired 4. The traces show why: after delegating, the writer immediately runs its own `grep` and `read` calls, follows the code's references and reaches the gold file itself, which is fast and almost free. Its final patch edited a gold file in 95% of extended runs whatever the explorer reported, and in all 30 Verified runs although Jev's and search's top three contained it only 7/10 and 6/10 times, including on Django's large codebase. Editing the right file is necessary (no patch that missed every gold file passed, 0 of 8) but not sufficient: even the oracle repaired no more issues, so with this writer the limiting step is writing the correct change, not finding the file. The saving therefore comes from not running a generative explorer whose work the writer largely repeats, not from the quality of Jev's choices, and the search control captures it equally. Jev's top score also did not separate correct from incorrect selections in the Verified sample (misses up to 0.85, hits as low as 0.66).

Table 3: File selection on the 225-issue SWE-bench development split, 12 candidates, one Jev decision per issue. W/L: issues where only Jev, or only search, chose a gold file.

| Repository | Issues | Recall@12 (%) | Search hit@3 (%) | Jev hit@3 (%) | W/L |
|---|---|---|---|---|---|
| marshmallow | 9 | 100.0 | 100.0 | 100.0 | 0/0 |
| pvlib | 63 | 95.2 | 88.9 | 93.7 | 4/1 |
| pydicom | 56 | 92.9 | 67.9 | 85.7 | 11/1 |
| astroid | 31 | 90.3 | 61.3 | 87.1 | 8/0 |
| pyvista | 16 | 75.0 | 43.8 | 68.8 | 5/1 |
| sqlfluff | 50 | 78.0 | 58.0 | 76.0 | 10/1 |
| All | 225 | 88.9 | 70.2 | 85.3 | 38/4 |

Table 4: File selectors on the same 225 issues: hit@3, decision latency, and cost per 1,000 decisions. Jev and the LLMs answer the identical per-file question; rows with 12 candidates see identical input.

| Selector | Hit@3 (%) | Median (s) | 90th pct. (s) | USD per 1,000 |
|---|---|---|---|---|
| Search only, top 3 of 12 | 70.2 | – | – | 0.00 |
| Jev, 12 candidates | 85.3 | 0.29 | 0.42 | 0.22 |
| Jev, 24 candidates | 90.2 | 0.34 | 0.44 | 0.39 |
| Jev, 48 candidates | 91.1 | 0.40 | 8.06 | 0.69 |
| gpt-6-luna, 12 candidates | 85.8 | 1.52 | 4.08 | 0.65 |
| GPT-6.1-Sol, 12 candidates | 88.0 | 1.82 | 2.20 | 12.59 |

**Q3 with a restricted writer: selection starts to matter.** When the writer may read and edit only the chosen files (Table 2, bottom), the ordering expected from selection quality appears: the oracle repaired 16, Jev 14 and search 11 of 41 issues. A selection that missed every gold file was now almost always fatal (0 of 5 Jev and 1 of 13 search runs passed, against 14 of 36 and 10 of 28 after a hit), and search runs took $-6.9$ s longer than Jev's on average (interval $-16.4$ to $1.6$) as the writer floundered. About 60% of runs in every condition attempted a refused read or search, so the restriction was exercised. With 4 wins and 1 loss for Jev over search (oracle over search 6/1), these directions are not yet significant on 41 issues. The run cost $10.80.

**Completed work, obstacles and next steps.** Completed: the development pilot, a Linux reproduction, the Verified pipeline with image checks, the 30-run Verified pilot, the 164-run extended run, the 123-run restricted-writer run, the file-selection study, sweep and LLM baselines (1,125 decisions), and 27 offline checks of method equivalence, leakage boundaries, fallbacks and budget denial; every reported number can be recomputed offline from the archives in the project repository (`https://github.com/KristjanDagur/CS329zFinalProject`), following its README, except the Verified pilot, whose run directory is still to be archived. Total spend is about $34 of OpenRouter credit and no Tinker credit. Next, the restricted writer will be applied to a shared 20-task Verified subset by both arms from one commit, and repeated rollouts will test whether its ordering is significant; we will add an empty-explorer control and a cheaper writer, which may recover less well, and use 24 candidates where latency allows. Five or ten single rollouts support no interval claims (2/5 has a 95% interval of roughly 12% to 77%), so the final study will use repeated rollouts and paired bootstrap intervals as in the extended run. We also plan to move Jev to a decision the writer cannot bypass, uncertainty-gated escalation between a cheap and a strong writer with thresholds chosen on independent development data, which returns to our proposal's calibrated-confidence question.

**Safety considerations.** We use public software issues, not clinical data or deployed systems; generated patches are research artifacts and are never merged upstream. Credentials never reach traces or agent processes. Verified tests run in network-disabled containers, but development tests run on the host, and the agent's edits are limited only by tool permissions; future runs should isolate every environment.

## REFERENCES

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*. https://arxiv.org/abs/2310.06770.

OpenAI. 2024. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/.

OpenCode contributors. OpenCode 1.19.5, native task/explore implementation. Software source. Accessed November 6, 2026. https://github.com/anomalyco/opencode/blob/v1.19.5/packages/opencode/src/tool/task.ts.

OpenRouter. Jev documentation: using the TypeSafe decision model on OpenRouter. API documentation. Accessed November 6, 2026. https://openrouter.ai/docs/guides/community/jev.

SWE-bench contributors. SWE-bench Lite dataset, development split. Revision 6ec7bb89b9342f664a54a6e0a6ea6501d3437cc2. Accessed November 6, 2026. https://huggingface.co/datasets/princeton-nlp/SWE-bench_Lite.

SWE-bench contributors. SWE-bench Verified dataset, test split. Revision 78f471bf655a3137b2e8a75af1501690ec009ec3. Accessed November 6, 2026. https://huggingface.co/datasets/SWE-bench/SWE-bench_Verified.

## AI-USE DISCLOSURE STATEMENT

**Rami.** OpenAI Codex assisted with interpreting the instructor feedback, inspecting source and documentation, implementing the provider adapter and local evaluator, executing the development pilot, and drafting the first version of this report. **Kristjan.** Anthropic's Claude (Claude Code, Opus 5.5) assisted with Linux reproduction, the API-key loading change, selecting the Verified sample, implementing and checking the Verified runner and analysis, interpreting the results, and revising this report. **Ómar.** Anthropic's Claude Code assisted with reviewing the repository and report, designing, implementing and running the file-selection study, the shortlist sweep the extended development run, the restricted-writer run and the LLM selector baselines (including the runner's oracle, timing, lane and restricted-writer options), analyzing their results, debugging the restricted-writer smoke run, merging the team's report versions, and writing this version of the report and the repository README. **Within the evaluated system.** Jev 1.14 made file-relevance decisions within the evaluated agent; GPT-6.1-Sol produced the experiments' patches and native exploration responses. Benchmark issues, upstream fixes, and tests come from SWE-bench rather than being generated for this project. The authors reviewed the reported results against the preserved run artifacts and take full responsibility for this report's contents.